

Московский авиационный институт
(национальный исследовательский университет)

Факультет информационных технологий и прикладной
математики

Кафедра вычислительной математики и программирования

Лабораторная работа №9 по курсу «Дискретный анализ»

Студент: Д. О. Кочев
Преподаватель: А. А. Кухтичев
Группа: М8О-306Б-22
Дата:
Оценка:
Подпись:

Москва, 2024

Лабораторная работа №9

Задача: Задан неориентированный граф, состоящий из n вершин и m ребер. Вершины пронумерованы целыми числами от 1 до n . Необходимо вывести все компоненты связности данного графа.

Формат ввода

В первой строке заданы $1 \leq n \leq 10^5$ и $1 \leq m \leq 10^5$. В следующих m строках записаны ребра. Каждая строка содержит пару чисел – номера вершин, соединенных ребром.

Формат вывода

Каждую компоненту связности нужно выводить в отдельной строке, в виде списка номеров вершин через пробел. Строки при выводе должны быть отсортированы по минимальному номеру вершины в компоненте, числа в одной строке также должны быть отсортированы.

1 Описание

Для решения задачи требуются знать самые основы работы с графами. В своем решении я использовал алгоритм обхода графа DFS, сложность которого $\Theta(V + E)$, где V - количество вершин в графе, E - количество ребер.

В начале я выбрал список смежности в качестве способа хранения графа. Затем я проходил по списку смежности и вызывал для каждой вершины, которая не посещалась ранее, алгоритм DFS. Если обход в глубину заканчивался, то значит все вершины, которые были пройдены за этот обход, образуют компоненту связности. Оставалось ее вывести и начать новый обход с вершины, которая раньше не посещалась. Мне удалось реализовать данное решение за $\Theta(n + m + n * C)$, где n - количество вершин в графе, m - количество ребер, а C - количество компонент связности.

2 Исходный код

Мой код получился небольшим и состоит из двух функций: *dfs* и *main*. Ниже показана глобальная вспомогательная переменная и обход в глубину.

```
1 int counter = 0;
2
3 void dfs(std::vector<std::vector<int>>& matrix, std::vector<int>& used, int node, std
   ::vector<int>& answer){
4     answer.push_back(node + 1);
5     used[node] = 1;
6     while (used[counter] == 1){
7         counter++;
8     }
9     for (int i = 0; i < matrix[node].size(); i++){
10         if (used[matrix[node][i]] == 0){
11             dfs(matrix, used, matrix[node][i], answer);
12         }
13     }
14 }
```

Функция *main*, где происходит чтение графа, его запись, и вызов функции *dfs*, пока все ребра не окажутся посещенными:

```
1 int main(){
2     int n, m;
3     std::cin>>n>>m;
4     std::vector<std::vector<int>> matrix(n);
5     for (int i = 0; i < m; i++){
6         int a, b;
7         std::cin>>a>>b;
8         matrix[a - 1].push_back(b - 1);
9         matrix[b - 1].push_back(a - 1);
10    }
11    std::vector<int> used(n);
12    while (counter != n){
13        std::vector<int> answer;
14        dfs(matrix, used, counter, answer);
15        sort(answer.begin(), answer.end());
16        for (int i = 0; i < answer.size(); i++){
17            std::cout<<answer[i]<<" ";
18        }
19        std::cout<<std::endl;
20    }
21 }
```

После завершения функции *dfs* выводятся отсортированные вершины одной компоненты связности.

3 Консоль

```
dmitrykochev$ cat test1.txt
5 4
1 2
2 3
1 3
4 5
dmitrykochev$ ./laba9 <test1.txt
1 2 3
4 5
dmitrykochev$ cat test2.txt
7 5
1 2
2 3
2 4
5 6
6 7
dmitrykochev$ ./laba9 <test2.txt
1 2 3 4
5 6 7
dmitrykochev$ cat test3.txt
6 3
1 2
2 3
3 4
dmitrykochev$ ./laba9 <test3.txt
1 2 3 4
5
6
```

4 Тест производительности

Тест производительности представляет из себя следующее: я замерю скорость своей программы на больших тестах и проверю, укладывается ли программа в заданное ограничение времени. Тестов будет три, они характеризуются количеством вершин и отрезков между ними. В первом тесте $n = 10^4$, $m = 5 * 10^3$. Во втором $n = 10^5$, $m = 5 * 10^4$. В третьем тесте $n = 10^5$, $m = 10^5$.

```
dmitrykochev$ ./benchmark <test1.txt
Algorithm time: 38293us
dmitrykochev$ ./benchmark <test1.txt
Algorithm time: 416921us
dmitrykochev$ ./benchmark <test1.txt
Algorithm time: 524173us
```

Как можно заметить, самый большой тест занимает половину секунды, что удовлетворяет ограничениям задачи. Тест производительности подтвердил, что алгоритм получился верным.

5 Выводы

Выполнив девятую лабораторную работу по курсу «Дискретный анализ», я узнал, что такое граф, как его хранить, и какие алгоритмы к нему можно применять. Эти знания являются необходимыми в работе программиста. Графы используются во многих прикладных задачах, так что данная лабораторная работа очень важна и полезна.

Список литературы

- [1] Томас Х. Кормен, Чарльз И. Лейзерсон, Рональд Л. Ривест, Клиффорд Штайн. *Алгоритмы: построение и анализ, 2-е издание.* — Издательский дом «Вильямс», 2007. Перевод с английского: И. В. Красиков, Н. А. Орехова, В. Н. Романов. — 1296 с. (ISBN 5-8459-0857-4 (рус.))